VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Computer Science Department

Midterm Progress Report

Paper: Adaptive Normalized Representation Learning for Generalizable Face Anti-Spoofing

Course: Pattern Recognition

Students:
Le Truong Thinh (23127018)
Le Minh Duc (23127351)

Instructors:
Vo Thanh Lam
Nguyen Thanh Tinh

March 1, 2026

Contents

1 Introduction **1**
1.1 Group information . . . 1
1.2 Project planning and Task distribution . . . 1
1.3 Abstraction . . . 1

2 Handbook of Biometrics **1**
2.1 Summarize the information in a book chapter . . . 1
2.1.1 Definition and scoping of biometrics . . . 1
2.1.2 Biometric access system flow . . . 2
2.1.3 Attack methods and spoofing detection . . . 2
2.2 Historical survey . . . 3
2.2.1 Fingerprint . . . 3
2.2.2 Iris . . . 4
2.2.3 Face (depth research) . . . 4
2.3 Core techniques and methods of Face anti-spoofing . . . 5
2.3.1 Motion detection . . . 5
2.3.2 Surface texture analysis - Skin texture and micro-pattern analysis . . . 9
2.3.3 Thermal imaging . . . 11
2.3.4 Frequency-domain (Fourier) analysis . . . 13
2.3.5 Challenge-response method . . . 15

3 Literature review **18**
3.1 Historical survey . . . 18
3.1.1 Attack Types . . . 18
3.1.2 Face Anti-Spoofing . . . 19
3.2 Related work . . . 21
3.2.1 Face Anti-Spoofing methods and techniques . . . 21
3.2.2 Batch-Instance Normalization . . . 24

4 Methodology Analysis **26**
4.1 Proposed method: Adaptive feature normalization . . . 26
4.1.1 Normalization . . . 26
4.1.2 Methodology . . . 27
4.2 Technical objectives and adopted approach . . . 28

4.2.1 Technical objectives 28
4.2.2 Adopted approach: Dual calibrated constraints (DCC) 29
4.3 Optimization strategy and base model supervision 29
4.3.1 Base model supervision: classification and depth estimation 29
4.3.2 Optimization via meta-learning 30
4.3.3 Effectiveness analysis 31

5 Conclusion **32**
5.1 Evaluation 32
5.1.1 Strengths 32
5.1.2 Limitations 32
5.1.3 Applicability 33
5.2 Improvements 34
5.2.1 Personal Insights on Effectiveness and Practical Relevance 34
5.2.2 Extensions 34

References **36**

1 Introduction

1.1 Group information

This report was written by the following four authors:

Student ID	Full name	Email
23127018	Le Truong Thinh	ltthinh23@clc.fitus.edu.vn
23127351	Le Minh Duc	lmduc23@clc.fitus.edu.vn

1.2 Project planning and Task distribution

Member	Main tasks	Complete
Thinh	Proposed methods of paper, technical evaluation, insights, future works and improvements	100%
Duc	Summary handbook, core techniques and algorithms in book, related works	100%

Table 1: Project planning and task distribution

1.3 Abstraction

2 Handbook of Biometrics

2.1 Summarize the information in a book chapter

2.1.1 Definition and scoping of biometrics

Biometrics are body measurements and calculations related to human characteristics and features. Based on chapter 20 in book, biometrics is defined as an automated method of verifying or recognizing the identity of a living person based on physiological or behavioral characteristics.

Biometric authentication (or realistic authentication) is used in computer science as a form of identification and access control. It is also used to identify individuals in groups that are under surveillance.

- Physiological characteristics (related to the shape of the body): fingerprint, palm veins, face recognition, DNA, palm print, hand geometry, iris recognition, retina, odor/scent, voice, shape of ears and gait.
- Behavioral characteristics: mouse movement, typing rhythm, gait, signature, voice, and behavioral profiling

The critical problem of verifying whether a presented biometric trait originates from a living human—a process known as liveness or spoof detection—has received comparatively less attention.

2.1.2 Biometric access system flow

1. **Acquisition**
   - The biometric is presented to the sensor by the person requesting access.
   - The raw biometric information is acquired and preprocessed, then it sent to the biometric feature extractor.
2. **Feature extraction**
   - Specialized software isolates the vital identifying features from the raw data.
   - This extracted dataset forms a digital representation called a template, which is subsequently sent to the matcher.
3. **Matching**
   - The matcher algorithm compares the newly presented template against previously stored templates to formulate a decision.
   - The system may compare the data against a single enrolled user for verification or against all enrolled users for identification.

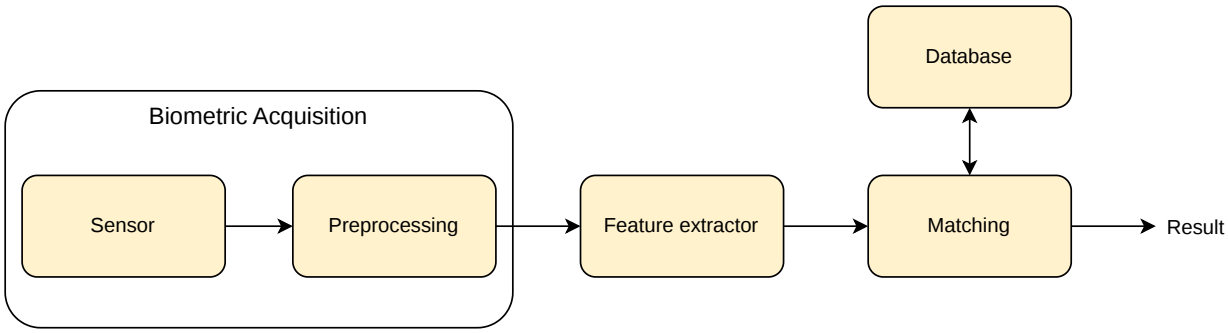

Figure 1: An example of how biometric data travels to obtain a result

2.1.3 Attack methods and spoofing detection

a. Attack Methods in Biometric Systems

1. **Presentation (Spoof) Attacks**

   A spoof attack occurs when a counterfeit biometric trait is presented directly to the sensor. Examples include:
   - Artificial fingerprint made from gelatin, silicone, latex, clay, or rubber.
   - Printed iris image or textured contact lens.
   - Face photograph or video replay.
   - 3D facial mask.

   The objective may be:
   - Identity concealment (avoiding recognition).

- Impersonation (gaining another user's privileges).
- Identity sharing (multiple users sharing an artificial biometric).

2. **Replay Attacks**
   Replay attacks intercept the communication channel between system components and inject previously captured genuine biometric data (raw image or extracted template).
3. **Database Attacks**
   The enrollment database may be:
   - Modified to enroll unauthorized users.
   - Altered to replace legitimate user templates.
4. **Trojan Horse Attacks**
   Core system modules may be replaced:
   - Feature extractor replaced with malicious software outputting predefined templates.
   - Matcher replaced with a module that always returns "match".

b. Spoofing Detection

Spoofing detection (or liveness detection) aims to determine whether the presented biometric originates from a live human rather than an artificial source.

Spoof detection can be implemented in two ways:

- **Decoupled systems**: Biometric capture occurs only after liveness is verified.
- **Integrated systems**: Spoof detection is performed during or after acquisition.

Detection approaches are generally grouped into three categories:

- **Static feature analysis**: Uses only the captured biometric data (e.g., texture or frequency analysis).
- **Dynamic/temporal analysis**: Detects physiological changes such as perspiration, blinking, or pupil movement.
- **Hardware-based methods**: Measures biological properties such as temperature, pulse, or spectral reflectance.

Performance is typically evaluated using metrics such as:

- **False Acceptance Rate** (FAR)
- **False Rejection Rate** (FRR)

2.2 Historical survey

2.2.1 Fingerprint

The earliest documented fingerprint spoofing attempt latent fingerprints could be forged using photographic and engraving techniques. A latent print was dusted, photographed, and etched onto a copper plate, enabling the reproduction of counterfeit fingerprints.

Modern fingerprint spoofing demonstrated that commercial fingerprint sensors could be deceived using artificial fingers made from soft materials such as gelatin, silicone, latex, clay, and rubber.
Fingerprint spoof fabrication generally follows two approaches:

- **Cooperative retrieval:** The legitimate user provides a mold using materials such as wax or dental impression compounds, from which artificial fingers are cast.
- **Non-cooperative retrieval:** Latent fingerprints are lifted from surfaces, digitized, enhanced, and used to create molds via etching or printing techniques.

Different fingerprint sensor technologies exhibit different vulnerabilities:

- **Optical (TIR-based) sensors:** Vulnerable to artificial fingers and latent prints due to reliance on reflectance differences.
- **Capacitive sensors:** Susceptible to conductive spoof materials that mimic skin capacitance.
- **Thermal sensors:** Can be deceived by materials approximating skin temperature.
- **Ultrasound sensors:** Vulnerable to soft materials with similar acoustic properties.

2.2.2 Iris

Iris recognition systems typically rely on near-infrared imaging and encode iris texture using phase-based feature extraction methods.
Spoofing attacks against iris systems commonly involve:

- High-resolution printed iris photographs.
- Iris images captured from sensors and reprinted.
- Textured contact lenses with printed iris patterns.
- Artificial three-dimensional iris structures.

To counter these attacks, several anti-spoofing mechanisms were introduced:

- Fourier spectrum analysis to detect printing artifacts.
- Detection of involuntary pupil oscillation (hippus).
- Light-induced pupil reflex testing.
- Challenge-response interactions such as blinking or gaze movement.
- Analysis of optical reflections from ocular structures.

2.2.3 Face (depth research)

Face recognition systems can be broadly categorized into: **two-dimensional (2D)** and **three-dimensional (3D)** systems.
Early 2D face recognition systems were highly vulnerable to spoofing via:

- Printed photographs.
- Video replay attacks displayed on screens.
- Digital image presentation via mobile devices or laptops.

- In poorly designed systems, even crude facial drawings.

More advanced spoofing methods include:

- Three-dimensional facial masks.
- Exploitation of identical twins.

To address these vulnerabilities, anti-spoofing approaches evolved toward **detecting motion** and **physiological cues**:

1. **Eye blinking detection**.
2. **Facial micro-movement analysis**.
3. **Skin texture and micro-pattern analysis**.
4. **Thermal imaging** to capture heat distribution.
5. **Frequency-domain (Fourier) analysis** to detect printed artifacts.
6. **Challenge-response prompts** (smile, blink, speak).

2.3 Core techniques and methods of Face anti-spoofing

2.3.1 Motion detection

a. Eye Blinking Detection

- **Definition**: Eye blinking detection in face anti-spoofing refers to the use of temporal changes in the appearance of a subject's eyes to determine liveness (printed photos, static masks, or simple video replays without blinking).
- **Core techniques and algorithms**

  Algorithm summary (CRF-based approach [14]):

  1. Detect face and extract eye ROI for each frame.
  2. Compute Eye Closity c_t from eye appearance features.
  3. Construct a Linear-chain CRF with unary and pairwise potentials.
  4. Perform sequence inference (Viterbi) to estimate eye states.
  5. Detect valid blink transition patterns.
  6. Decide liveness based on blink occurrence.

  1. Face and eye region detection

  1. For each frame $I_t(x, y)$, detect the face region:

$$B_t = \text{FaceDetector}(I_t(x, y)).$$

  2. Extract the eye **region of interest (ROI)** within B_t.
  3. Normalize the eye patch to a fixed size to reduce scale variation.

  2. Eye Closity estimation

  Instead of using geometric ratios, the method defines a continuous measure called **Eye Clos-**

ity, representing the probability of the eye being closed.

For each frame t, extract appearance features $\mathbf{x}_t$ from the eye ROI (e.g., intensity, texture descriptors).

A trained classifier produces:

$$c_t = P(y_t = \text{closed} \mid \mathbf{x}_t),$$

where:

- $c_t \in [0, 1]$ is the Eye Closity value,
- y_t is the hidden eye state (open or closed).

Eye Closity provides a soft, probabilistic representation rather than a hard threshold.

3. Temporal modeling with Linear-chain CRF

Eye blinking is treated as a structured temporal sequence problem. Let:

$$\mathbf{y} = \{y_1, y_2, \ldots, y_T\}$$

be the hidden eye states over T frames, and

$$\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}$$

be the observed features.

A Linear-chain CRF models the conditional probability[15]:

$$P(\mathbf{y} \mid \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp\left(\sum_t \psi_u(y_t, \mathbf{x}_t) + \sum_t \psi_p(y_{t-1}, y_t)\right),$$

where:

- ψ_u is the unary potential (related to Eye Closity),
- ψ_p is the pairwise potential modeling state transitions,
- $Z(\mathbf{x})$ is the normalization factor.

The CRF captures temporal dependencies and enforces realistic blinking dynamics (open $\rightarrow$ closed $\rightarrow$ open).

4. Blink sequence inference

The optimal state sequence is obtained by:

$$\mathbf{y}^* = \arg\max_{\mathbf{y}} P(\mathbf{y} \mid \mathbf{x}),$$

typically solved using the Viterbi algorithm.

A blink event is defined as a valid transition pattern:

$$\text{open} \rightarrow \text{closed} \rightarrow \text{open}.$$

5. Decision logic for liveness

Count the number of valid blink patterns N_{blinks} within the observation window.

$$\text{decision} = \begin{cases} \text{live} & \text{if } N_{\text{blinks}} \geq N_{\min}, \\ \text{spoof} & \text{otherwise.} \end{cases}$$

- **Advantages**
  - Models temporal dependencies explicitly.
  - Robust to noise and short-term misclassification.
  - Produces smoother and more realistic blink sequences.
- **Weaknesses**
  - Vulnerable to high-quality replay attacks containing natural blinking.
  - Requires sequence training data and CRF parameter estimation.
  - Performance may degrade under extreme head pose or occlusion.

b. Facial micro-movement analysis

- **Definition**: Facial micro-movement analysis in face anti-spoofing refers to the exploitation of subtle, involuntary, high-frequency facial motions (e.g., minute muscle contractions, micro-expressions, eye and lip micro-twitches) as liveness cues to distinguish real human faces from presentation attacks (e.g., printed photos, replay videos, masks).
- **Core techniques and algorithms** :

1. **Face tracking and alignment**

   Given an input video sequence $V = \{I_t\}_{t=1}^{T}$, perform face detection and alignment to stabilize facial region across frames:

$$B_t = \text{FaceDetector}(I_t), \quad F_t = \text{AlignFace}(I_t, B_t), \tag{1}$$

   where B_t is the bounding box and F_t the aligned face at time t.

2. **Region of Interest (ROI) extraction** [1]

   Define ROIs on dynamic facial regions (e.g., eyes, cheeks, mouth). Using Facial Action Coding System (FACS) regions simplifies grouping of facial muscles into action units (AUs) that exhibit meaningful micro-movements. These ROIs can be denoted as:

$$R_t^{(i)} = \text{ROIExtract}(F_t, \text{AU}_i), \quad i = 1, \ldots, n, \tag{2}$$

where AU_i is the facial action region index.

3. **Spatiotemporal feature computation (Dynamic Texture / LBP-TOP)**
   For each ROI sequence $\{R_t^{(i)}\}_{t=1}^T$, compute dynamic texture features based on Local Binary Patterns on Three Orthogonal Planes (LBP-TOP) [2]:

$$\text{LBP-TOP}(x, y, t;\, (R_t^{(i)})) = \{\text{LBP}_{XY},\, \text{LBP}_{XT},\, \text{LBP}_{YT}\},$$

   where LBP codes on planes XY, XT, and YT capture spatial and temporal transition patterns. The histogram of such codes over the ROI is:

$$H^{(i)} = \sum_{x,y,t} (\text{LBP-TOP}(x, y, t) \;\mapsto\; \text{bin indexing})\,. \tag{3}$$

   Dynamic texture features represent subtle temporal micro-movements that differ between live and spoof inputs.

4. **Optical flow based micro motion signature**
   An alternative way to detect micro-motions is to compute optical flow $\mathbf{u}_t(x, y) = (u_{x,t}, u_{y,t})$ between consecutive aligned frames:

$$\mathbf{u}_t(x, y) = \arg\min_{\mathbf{u}} \|F_{t+1}(x + u_x, y + u_y) - F_t(x, y)\|^2, \tag{4}$$

   Small flow magnitudes at specific facial landmarks (e.g., eyelids) indicate micro movements. Aggregate features can be derived as:

$$\mathbf{f}^{(i)} = \frac{1}{|R^{(i)}|} \sum_{(x,y)\in R^{(i)}} \|\mathbf{u}_t(x, y)\|. \tag{5}$$

5. **Temporal discriminative modeling**
   Stack extracted features over time into a feature vector:

$$\mathbf{F}_{\text{seq}} = \left[H^{(1)}, \ldots, H^{(n)},\, \mathbf{f}^{(1)}, \ldots, \mathbf{f}^{(n)}\right]$$

   and model the temporal evolution using a classifier (e.g., SVM, HMM) or threshold decision rule to decide liveness:

$$\hat{y} = \begin{cases} \text{live}, & g(\mathbf{F}_{\text{seq}}) \geq \tau, \\ \text{spoof}, & g(\mathbf{F}_{\text{seq}}) < \tau, \end{cases}$$

   where $g(\cdot)$ is a learned score function and τ a threshold.

- **Strengths**
  - Exploits genuine involuntary facial dynamics that are difficult for spoof mediums (flat photos, replay videos) to replicate.
  - Does not require specialized depth sensors or IR hardware and provide richer discriminative information than static texture alone.
- **Weaknesses**
  - Requires sufficiently high frame rate and stable alignment to reliably capture micro-motions and sensitive to motion blur and illumination changes across frames.
  - Some spoof attacks (e.g., high-quality replay with similar micro dynamics) can still mimic temporal patterns, reducing discriminative margin.

2.3.2 Surface texture analysis - Skin texture and micro-pattern analysis

- **Definition**: Surface texture analysis in face anti-spoofing refers to the examination of fine skin surface properties, micro-texture patterns, and local structural regularities that differentiate real human skin from spoof mediums (e.g., printed photos, screen displays).
- **Core techniques and algorithms** :
  1. **Face Region Localization and Pre-processing**
     Given an input image I, the first step is to detect and extract the face region:

     $$B = \text{FaceDetector}(I), \quad F = \text{Crop}(I, B), \tag{6}$$

     where B is the face bounding box and F is the cropped facial patch. Optionally apply **histogram equalization** to normalize illumination:

     $$F' = \text{HistEq}(F). \tag{7}$$

  2. **Local Texture Feature Extraction**
     The core of surface texture analysis is to extract local texture descriptors that capture micro-patterns in intensity variations.
     - **Local Binary Patterns (LBP)** [8] — a fundamental texture descriptor:

       $$\text{LBP}_{P,R}(x, y) = \sum_{p=0}^{P-1} s(I_p - I_c)2^p, \tag{8}$$

       where $I_c = F'(x, y)$ is the central pixel, I_p are P neighbors on a circle of radius R,

and

$$s(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0. \end{cases}$$

The LBP pattern encodes the micro-structure of local intensity changes. For an entire face patch, build a histogram of codes:

$$H_{LBP} = \text{Histogram}\big(\{\text{LBP}_{P,R}(x,y)\}\big). \tag{9}$$

3. **Frequency-Domain Texture Analysis**
   To capture periodic artifacts (e.g., printing halftone, screen pixel structures), analyze the face patch in frequency domain:

$$\mathcal{F}(u,v) = \text{DFT}\{F'(x,y)\}, \tag{10}$$

   where $\mathcal{F}(u,v)$ is the discrete Fourier transform at frequency (u,v). The magnitude spectrum:

$$M(u,v) = |\mathcal{F}(u,v)|, \tag{11}$$

   is analyzed for peaks or patterns indicating artificial periodic structures. For printed or screen spoofs, regular grid patterns produce prominent frequency spikes.
4. **Multi-Scale/Orientation Filters (Gabor)** [8]
   Gabor filters model localized spatial frequencies and orientations, capturing texture at multiple scales:

$$G_{\mu,\nu}(x,y) = \exp\Big(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\Big)\cos\Big(2\pi\frac{x'}{\lambda} + \phi\Big), \tag{12}$$

   where $x' = x\cos\theta_\nu + y\sin\theta_\nu$, $y' = -x\sin\theta_\nu + y\cos\theta_\nu$, μ is scale index, ν is orientation index. The filtered responses:

$$R_{\mu,\nu}(x,y) = (F' * G_{\mu,\nu})(x,y), \tag{13}$$

   form feature maps whose statistics (mean, variance) across the face patch form the feature vector.
5. **Feature Vector Assembly**
   Concatenate texture descriptors into a single feature vector:

$$\mathbf{f} = [H_{LBP},\, \mathbf{g}_{\text{Gabor}},\, \mathbf{h}_{\text{Freq}}], \tag{14}$$

where $\mathbf{g}_{\text{Gabor}}$ are Gabor statistics (e.g., at multiple scales/orientations), and $\mathbf{h}_{\text{Freq}}$ represents frequency domain signature.

6. **Classification / Decision**
   Use a classifier (e.g., Support Vector Machine) or distance threshold to separate real versus spoof. Real faces typically exhibit skin micro-texture patterns, while spoof images yield texture distributions with printing/screen artifacts.

- **Strengths**
  - Captures fine textural differences between real skin and artificial mediums (e.g., printing dots, display pixel grids).
  - Works on single images — no requirement for temporal sequences or special hardware.
- **Weaknesses**
  - Sensitive to illumination variations; requires good pre-processing.
  - Texture descriptors may be fooled by high-quality spoof mediums with strong texture fidelity and it dependent on image resolution — too low resolution loses micro-patterns.

2.3.3 Thermal imaging

- **Definition**: Thermal imaging for face anti-spoofing refers to the use of cameras sensitive to long-wave infrared (LWIR, typically 8–14 μm) to capture temperature distributions of a subject's face.
- **Core techniques and algorithms**
  Algorithm summary:
  1. Acquire thermal image I_T; convert radiance to temperature map $T(x, y)$. [16]
  2. Detect ROI and normalize temperature and extract texture (LBP), gradient, and statistical features from ROI.
  3. Train SVM/Bayesian classifier on feature vectors.

  1. Preprocessing

  (a) *Temperature normalization*

  Thermal cameras produce raw pixel values proportional to radiance; these must be converted to scene temperature $T(x, y)$ using calibration:

$$L(x, y) = \epsilon\sigma T(x, y)^4,$$

  where L is radiance, ϵ is emissivity, and σ is the Stefan–Boltzmann constant. Solve for $T(x, y)$ to linearize measurements.

  (b) *Face region of interest (ROI) detection*

  Localize facial thermal patterns, often via classical segmentation (thresholding) or alignment

using visible–thermal co-registration when combined sensors exist:

$$\text{ROI} = \{(x, y) : T(x, y) > \tau\},$$

where τ is an adaptive threshold determined from histogram analysis of the thermal image.

2. Feature extraction

Thermal anti-spoofing uses features that capture statistical and spatial temperature distributions.

(a) *Thermal texture descriptors*

Raw temperature maps $T(x, y)$ exhibit different spatial frequency content for live faces vs. spoof attacks. One classical descriptor is **Local Binary Patterns (LBP)** applied to thermal:

$$\text{LBP}(x_c, y_c) = \sum_{n=0}^{P-1} s(T_n - T_c)2^n,$$

where $s(z) = 1$ if $z \geq 0$ else 0, T_c is center pixel and $\{T_n\}$ are neighbors.

(b) *Thermal gradient features* [16]

Compute gradients $\nabla T = \left(\frac{\partial T}{\partial x}, \frac{\partial T}{\partial y}\right)$ to capture edges. Gradient magnitudes differ for human skin vs. printed/spoof surfaces:

$$M(x, y) = \sqrt{\left(\frac{\partial T}{\partial x}\right)^2 + \left(\frac{\partial T}{\partial y}\right)^2}.$$

Statistics (mean, variance) of M over ROI form feature vectors.

(c) *Physiological models*

Humans exhibit characteristic thermal distribution patterns due to facial blood perfusion:

$$T(x, y) = T_{\text{ambient}} + H(x, y),$$

where $H(x, y)$ approximates perfusion-dependent heat. Statistical measures like PCA (Principal Component Analysis) or Fisherfaces can capture dominant modes:

$$\mathbf{F} = \arg\min_{\mathbf{W}} \sum_i \|\mathbf{W}^\top(\mathbf{t}_i - \bar{\mathbf{t}})\|^2,$$

where $\mathbf{t}_i$ are thermal feature vectors.

3. Classification (Using SVM or Bayesian classifiers)

(a) *Support Vector Machines (SVM)*

For linearly separable features $\mathbf{f}_i$ with labels $y_i \in \{-1, 1\}$,

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{f}_i + b) \geq 1.$$

Kernelized variants (e.g., RBF) handle nonlinearity typical in thermal vs. spoof distributions.

(b) *Bayesian classifiers*

Estimate class-conditional densities $p(\mathbf{f}|c)$ for live vs. spoof and apply Bayes decision:

$$\hat{c} = \arg \max_{c \in \{\text{live,spoof}\}} p(\mathbf{f}|c)p(c).$$

Gaussian models with mean $\boldsymbol{\mu}_c$ and covariance Σ_c are common.

- **Strengths**
  - Thermal signatures are inherently related to physiological heat emission, making them difficult for static print or display attacks to replicate.
  - Works under illumination conditions where visible-spectrum methods fail.
  - Thermal gradients capture perfusion patterns that are invariant to color or makeup.
- **Weaknesses**
  - Some 3D masks or heated spoof devices can approximate thermal patterns, reducing effectiveness.
  - Thermal resolution is often lower than visible imaging, complicating fine-grained analysis.

2.3.4 Frequency-domain (Fourier) analysis

- **Definition**: Frequency-domain (Fourier) analysis in face anti-spoofing refers to transforming face images from the spatial domain into the frequency domain using Fourier-based transforms to reveal underlying periodic patterns.
- **Core techniques and algorithms**

  Algorithm summary (foundation):
  1. Detect and normalize face region $I(x, y)$.
  2. Convert to grayscale (optional) and window the image to reduce boundary artifacts.
  3. Compute 2D DFT $F(u, v)$ and log-magnitude spectrum.
  4. Derive radial and directional spectral features: $E(r)$ and $E_\theta(\rho)$.
  5. Form a frequency feature vector $\mathbf{f}$ (mean/variance/peaks).
  6. Train a classifier (e.g., SVM) on $\mathbf{f}$; classify test samples.

  1. Preprocessing

  (a) *Face detection & normalization*

  Detect face region $I(x, y)$ from the original RGB frame. Crop and resize to a fixed size $N \times N$

to standardize the Fourier analysis:

$$I_{\text{norm}}(x, y) = I\big(x + x_0, y + y_0\big) \quad \text{for } 0 \le x, y < N.$$

(b) *Grayscale conversion (optional)*
Many methods work on luminance $L(x, y)$ to simplify computation:

$$L(x, y) = 0.299R + 0.587G + 0.114B.$$

2. Fourier transform
Compute the 2D **Discrete Fourier Transform (DFT)** [11] of the normalized face patch $f(x, y)$:

$$F(u, v) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \mathrm{e}^{-j2\pi\left(\frac{ux+vy}{N}\right)},$$

where (u, v) are frequency coordinates, and $j = \sqrt{-1}$. The magnitude spectrum is:

$$|F(u, v)| = \sqrt{\Re(F)^2 + \Im(F)^2}.$$

Often a log-scale spectrum $S(u, v) = \log(1 + |F(u, v)|)$ is computed to compress dynamic range.

3. Spectral analysis
(a) *Radial frequency energy distribution*
Compute radial averages to capture energy distribution across frequency radii $r = \sqrt{u^2 + v^2}$. Partition the spectrum into concentric rings and compute average energy per ring:

$$E(r) = \frac{1}{|\mathcal{R}_r|} \sum_{(u,v) \in \mathcal{R}_r} S(u, v),$$

where $\mathcal{R}_r = \{(u, v) : r - \Delta/2 < \sqrt{u^2 + v^2} \le r + \Delta/2\}$ and Δ is ring thickness. Live faces tend to show smoother, broadband distributions, whereas spoof prints/screens have spikes at specific frequencies (halftone dots or pixel grid patterns).
(b) *Directional frequency features*
To capture periodic patterns orientations (e.g., horizontal/vertical periodic artifacts in displays):

$$E_\theta(\rho) = \sum_{\substack{(u,v): \\ \angle(u,v) \approx \theta}} S(u, v),$$

where $\angle(u, v) = \arctan(v/u)$. Compute features at $\theta = \{0°, 45°, 90°, 135°\}$.

4. Feature extraction

Construct feature vector $\mathbf{f}$ from selected statistical measures of the frequency spectrum:

- Mean and variance of energy in low, mid, high frequency bands:

$$\mu_b = \frac{1}{|\mathcal{B}_b|} \sum_{(u,v)\in\mathcal{B}_b} S(u, v), \quad \sigma_b^2 = \frac{1}{|\mathcal{B}_b|} \sum_{(u,v)\in\mathcal{B}_b} \big(S(u, v) - \mu_b\big)^2,$$

  where $\mathcal{B}_b$ is a frequency band ($b \in \{\text{low}, \text{mid}, \text{high}\}$).
- Peaks in frequency bands indicative of periodic spoof artifacts:

$$p_b = \max_{(u,v)\in\mathcal{B}_b} S(u, v).$$

5. Classification

Feed the feature vector $\mathbf{f}$ into a classical classifier such as Support Vector Machine (SVM), Random Forest, or k-Nearest Neighbors (kNN) to decide live vs. spoof.

- **Strengths**
  - Can reveal spoof-specific periodic patterns (moire from printing, pixel grid from screens) not obvious in spatial domains.
  - Frequency features are translation-invariant and robust to small spatial misalignments.
  - Computationally efficient via fast FFT implementations.
- **Weaknesses**
  - Sensitive to image resolution and quality; low-resolution inputs blur spectral distinctions.
  - Can be fooled by high-quality prints/screens designed to suppress periodic artifacts.
  - Works best for print/screen attacks; less effective for 3D mask attacks without clear spectral signatures.

2.3.5 Challenge-response method

- **Definition**: Challenge-response methods in face anti-spoofing are interactive liveness checks in which the system issues a "challenge" to the subject (e.g., ask the user to blink, turn the head, open the mouth, or follow a pattern), then verifies that the subject's responses match expected temporal and spatial behavior.
- **Core techniques and algorithms**

  Algorithm summary:
  1. System selects a challenge sequence $C = \{c_t\}_{t=1}^{T}$.
  2. Capture video frames $I = \{I_t(x, y)\}$ while user performs challenge.
  3. Detect facial landmarks L_t in each frame.

4. Extract action features (EAR, MAR, head pose) per frame to form $A = \{a_t\}$.
5. Compare observed action sequence A with expected $\hat{A}$ and compute similarity score.
6. Apply threshold to determine live vs. spoof.

1. Challenge design

The system selects a challenge from a set of predefined challenge types. Let the challenge sequence be

$$C = \{c_t\}_{t=1}^{T},$$

where each c_t is an expected action at time t (e.g., blink, head turn). A typical challenge might be:

$$C = [\text{blink},\ \text{turn left},\ \text{smile}].$$

The challenge duration T is chosen to balance usability and security.

2. Video capture & preprocessing

Acquire a video sequence

$$I = \{I_t(x, y)\}_{t=1}^{T},$$

where $I_t(x, y)$ is the frame at time t. Detect face position and facial landmarks (eyes, mouth, nose) in each frame using a detector (e.g., Viola-Jones or Active Shape Models circa 2010):

$$L_t = \{(x_i^t, y_i^t)\}_{i=1}^{n},$$

where each (x_i^t, y_i^t) is a landmark coordinate.

3. Feature extraction for action verification

For each expected action c_t, extract features that quantify the corresponding motion:

(a) *Eye blink detection*

Compute eye aspect ratio (EAR) [18] from eye landmarks:

$$\text{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|},$$

where $p_1..p_6$ are eye landmarks. A blink is detected when EAR drops below a threshold τ_{blink}:

$$\text{blink}_t = \begin{cases} 1 & \text{if } \text{EAR}_t < \tau_{\text{blink}}, \\ 0 & \text{otherwise.} \end{cases}$$

(b) *Head pose estimation*

Estimate 3D head rotation using a Perspective-n-Point (PnP) solver: given 2D landmark

positions $\{(x_i, y_i)\}$ and corresponding 3D model points $\{X_i\}$, solve

$$s \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \begin{bmatrix} X_i \\ 1 \end{bmatrix},$$

where $\mathbf{R}$ is rotation, $\mathbf{t}$ is translation, $\mathbf{K}$ is camera intrinsics, and s is a scale. The head yaw angle θ_{yaw} is extracted from $\mathbf{R}$.

(c) *Mouth open/close*

Compute mouth aspect ratio (MAR) [3]:

$$\text{MAR} = \frac{\|q_3 - q_9\| + \|q_4 - q_8\| + \|q_5 - q_7\|}{2\|q_1 - q_{11}\|},$$

where q_i are mouth landmarks. A mouth open event is detected when MAR exceeds a threshold τ_{mouth}.

4. Temporal modeling & matching

For each time t, produce an observed action signal a_t based on features:

$$a_t = f_{\text{feat}}(L_t),$$

where f_{feat} maps landmark positions to a binary or continuous signal for the expected action. For the sequence,

$$A = \{a_t\}_{t=1}^{T}.$$

The system defines an expected response pattern $\hat{A} = \{\hat{a}_t\}_{t=1}^{T}$ corresponding to the challenge C. Compute a similarity measure between observed and expected:

$$\text{score} = \frac{1}{T} \sum_{t=1}^{T} \mathbb{I}(a_t = \hat{a}_t),$$

where $\mathbb{I}$ is the indicator function. Alternatively, continuous signals can be compared using normalized cross-correlation.

5. Decision logic

Decide liveness if the similarity score exceeds a threshold:

$$\text{decision} = \begin{cases} \text{live} & \text{if score} \geq \tau_{\text{decision}}, \\ \text{spoof} & \text{otherwise.} \end{cases}$$

Thresholds $\tau_{\text{blink}}, \tau_{\text{mouth}}, \tau_{\text{decision}}$ are chosen empirically or via training on labeled data.

- **Strengths**
  - Robust against static spoof attacks (printed photos, static masks) because spontaneous motion is required.
  - Intuitive and explainable (blinks, head movements are direct indicators of liveness).
  - Can be implemented on commodity cameras without specialized hardware.
- **Weaknesses**
  - Vulnerable to high-quality video replays where the spoof shows the expected actions.
  - Requires user cooperation and compliance, which can degrade user experience.
  - Performance depends on accurate landmark tracking; poor lighting or occlusions reduce reliability.
  - Challenge design and threshold tuning can be brittle across demographics or environmental conditions.

3 Literature review

3.1 Historical survey

The evolution of FAS techniques can be broadly categorized into three main stages: (1) handcrafted feature-based methods, (2) deep learning-based methods, and (3) generalization-oriented and multimodal approaches. [12] [9]

3.1.1 Attack Types

1. **Print Attacks (Early 2000s)**
   - An attacker presents a printed photograph of a target face to the camera.
   - These attacks are typically 2D and lack depth and motion consistency.
   - Early datasets: **NUAA Photograph Imposter Database** (2010) focused primarily on this attack type.
2. **Replay (Video) Attacks (2010–2013)**
   - With the proliferation of high-resolution displays and smartphones, replay attacks became prevalent.
   - Replay attacks introduce temporal dynamics, making them more challenging to detect than print attacks.
3. **3D Mask Attacks (2011–2015)**
   - 3D mask attacks involve the use of silicone, resin, or paper masks that replicate the geometry of a real face.
   - These attacks simulate depth and partial reflectance properties, making them significantly

harder to detect.

- Datasets such as **3DMAD** were introduced to evaluate robustness against such attacks.

4. Makeup and Partial Attacks (2016–2019)

- Advanced attacks include makeup-based obfuscation and partial attacks (e.g., eye-region manipulation).
- These attacks aim to bypass systems by altering specific facial regions while retaining realism.

5. Deepfake and AI-Generated Attacks (2018–Present)

- With the rise of generative models such as GANs, attackers can synthesize highly realistic face images and videos.
- Deepfake-based attacks introduce both spatial and temporal coherence, posing new challenges for FAS systems.

3.1.2 Face Anti-Spoofing

1. Handcrafted Feature-Based Methods (2007–2015)

Early FAS approaches relied on manually designed features derived from texture, motion, and frequency analysis.

- *Texture-Based Methods*: **Local Binary Patterns (LBP)** and its variants (e.g., **LBP-TOP**) were widely used to capture micro-texture differences between live skin and printed surfaces.
- *Motion-Based Methods*: Techniques based on **optical flow** and **eye-blinking detection** were proposed to detect liveness cues (since printed photos lack natural motion patterns).
- *Frequency and Reflectance Analysis*: **Fourier spectrum analysis** and **image distortion** features were used to detect patterns or display artifacts in replay attacks.

Limitations: Handcrafted methods often failed to generalize across datasets due to variations in illumination, camera quality, and attack media.

2. CNN-Based Deep Learning Methods (2015–2018)

With the success of **Convolutional Neural Networks (CNNs)** in computer vision, FAS shifted toward data-driven approaches.

- *Binary Classification Models*: Early **CNN-based FAS methods** treated spoof detection as a binary classification problem (live vs. spoof). Pretrained architectures such as **VGG** and **ResNet** were fine-tuned on FAS datasets.
- *Patch-Based Learning*: To capture local spoof cues, some methods divided facial images into patches and trained CNNs on local regions.

Advantages: Improved feature representation and higher accuracy compared to handcrafted methods.

Drawback: Limited cross-dataset generalization.

3. Depth and Auxiliary Supervision Methods (2018–2020)

Researchers observed that spoof faces lack realistic depth information. Consequently, methods incorporating auxiliary supervision were proposed.

- *Depth Map Estimation*: Models were trained to regress pseudo-depth maps, where live faces exhibit 3D structure and spoof faces appear flat.
- *Reflection and rPPG Signals*: **Remote photoplethysmography (rPPG)** signals were used to capture subtle blood flow changes in real faces. These physiological signals are absent in spoof media.

This stage significantly improved robustness by introducing physical priors into deep models.

4. Domain Generalization and Meta-Learning (2019–2023)

Cross-dataset performance became a critical challenge. Models trained on one dataset often failed on unseen domains.

- *Domain Adaptation*: Adversarial learning and feature alignment techniques were introduced to reduce domain discrepancy.
- *Domain Generalization*: Meta-learning frameworks simulated domain shifts during training to improve unseen-domain robustness.
- *Style Randomization and Data Augmentation*: Methods such as style transfer and frequency augmentation were proposed to enhance invariance to environmental variations.

5. Transformer-Based and Multimodal Methods (2022–Present)

Recent research explores Vision Transformers (ViT) and hybrid CNN-Transformer architectures to model global dependencies.

- *Transformer Architectures*: Transformers capture long-range spatial and temporal dependencies, improving detection of sophisticated attacks.
- *Multimodal Fusion*: Combining RGB, depth, infrared (IR), and thermal modalities enhances robustness against complex attacks.
- *Self-Supervised and Foundation Models*: Large-scale pretraining and contrastive learning approaches aim to learn generalized spoof representations with limited labeled data.

Summary of Evolution

- 2007–2015: Handcrafted texture and motion-based methods.
- 2015–2018: CNN-based binary classifiers.
- 2018–2020: Auxiliary supervision (depth, rPPG).
- 2019–2023: Domain generalization and meta-learning.
- 2022–Present: Transformer-based, multimodal, and self-supervised approaches.

Table 2: Chronological comparison of representative RGB-based face anti-spoofing methods

Year	Method	Core Idea	Target	Limitation
2004	Frequency-based motion descriptor (FDD)	Analyze temporal frequency energy variations from facial motion	Photo attacks	Illumination sensitive; fails on replay
2005	Optical Flow Lines (OFL)	Compare motion between inner/outer facial regions (3D vs planar)	Photo attacks	Fails on replay; lighting sensitive
2007	Eye-blinking (CRF)	Sequential modeling of eye open/close states	Static photo	Vulnerable to replay and cut-photo
2007–2009	Interactive motion-based	Challenge–response (rotation, blink, speech)	Photo, replay	Intrusive; bypassed by prerecorded/synthetic video
2010–2013	Handcrafted texture (LBP, HOG, IQA)	Micro-texture differences of real vs spoof surfaces	Photo, replay	Poor generalization; weak vs 3D masks
2013–2015	Dynamic texture (LBP-TOP)	Spatio-temporal texture across frames	Photo, replay	Higher cost; weak vs realistic masks
2016	rPPG-based	Detect periodic RGB changes from blood flow	Photo, masks, low replay	Struggles with high-quality replay
2016–2018	CNN spatial modeling	Deep feature learning replaces handcrafted features	Photo, replay	Overfitting; weak cross-dataset
2018–2019	CNN + temporal (CNN-LSTM)	Joint spatial-temporal learning	Photo, replay	High computation; unseen attack weakness
2019–2021	Pseudo-depth supervision	Estimate depth to detect planar surfaces	Photo, replay	Limited vs realistic 3D masks
2020–2023	Generalization-oriented methods	Domain adaptation, NAS, few-shot learning	Focus on unseen/unknown attacks	Still challenged by 3D mask attacks and DeepFake
2022–present	Transformer-based and multimodal approaches	Vision Transformers, cross-modal learning, self-supervised pretraining	Improved robustness to unseen attacks	Remain challenged by high-quality 3D masks and advanced DeepFake attacks

3.2 Related work

3.2.1 Face Anti-Spoofing methods and techniques

Face Anti-Spoofing (FAS) has evolved from handcrafted-feature based approaches to deep learning and domain generalization frameworks. Below we summarize representative techniques and briefly describe their core ideas and algorithms.

Local Binary Pattern (LBP)

- **Idea:** Spoof faces exhibit different micro-texture statistics compared to live faces due to printing artifacts or display noise.
- **Algorithm:**
  - For each pixel, compare it with its P neighbors on a circle of radius R:

$$\mathrm{LBP}(x_c) = \sum_{p=0}^{P-1} s(x_p - x_c)2^p,$$

    where

$$s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

  - Convert LBP codes into histograms.
  - Use histograms as input features for a classifier.

LBP-TOP (Spatio-temporal LBP)

- **Idea:** Real faces contain natural temporal dynamics that spoof media lack.
- **Algorithm:**
  - Construct a video volume.
  - Compute LBP features on three orthogonal planes (XY, XT, YT).
  - Concatenate histograms from all planes.
  - Perform classification based on combined features.

Histogram of Oriented Gradients (HOG)

- **Idea:** Gradient and edge distributions differ between real faces and spoof artifacts.
- **Algorithm:**
  - Compute image gradients.
  - Divide the image into small cells.
  - Build histograms of gradient orientations per cell.
  - Normalize histograms across blocks.
  - Concatenate all histograms for classification.

SIFT-based Methods

- **Idea:** Local invariant keypoint descriptors can capture spoof inconsistencies.
- **Algorithm:**
  - Detect scale-space extrema.

  - Assign orientation to each keypoint.
  - Extract 128-dimensional descriptors.
  - Aggregate descriptors for classification.

CNN-based Binary Classification

- **Idea:** Learn discriminative spoof cues directly from RGB images via deep neural networks.
- **Algorithm:**
  - Given input image x, predict label:

$$\hat{y} = f_\theta(x), \quad \mathcal{L}_{cls} = -\sum y \log \hat{y}$$

  - Train the CNN end-to-end using cross-entropy loss.

Depth-based Auxiliary Supervision

- **Idea:** Live faces have 3D structure while spoof faces are planar.
- **Algorithm:**
  - Generate pseudo-depth maps for live faces and zero maps for spoof faces.
  - Train with regression loss:

$$\mathcal{L}_{depth} = \|D(E(x)) - d\|_2^2$$

  - Jointly optimize depth and classification losses.

rPPG-based Methods

- **Idea:** Real faces exhibit physiological signals (heartbeat-induced color changes) absent in spoof attacks.
- **Algorithm:**
  - Extract temporal facial signals from video.
  - Estimate periodic pulse patterns.
  - Classify based on signal consistency.

Disentanglement-based Methods

- **Idea:** Separate spoof-specific features from identity or domain-related features.
- **Algorithm:**
  - Encode image into latent representation.
  - Apply feature decomposition constraints.
  - Enforce reconstruction and classification consistency to isolate spoof traces.

Central Difference Convolution (CDCN)

- **Idea:** Spoof artifacts are more visible in high-frequency components.
- **Algorithm:**
  - Modify standard convolution as:

$$\mathrm{CDC}(x) = \mathrm{Conv}(x) - \theta \cdot \mathrm{Conv}(x_{center})$$

  - Enhance gradient sensitivity through central difference term.

Domain Generalization (DG) Methods

- **Multi-domain Adversarial Alignment**
  - **Idea:** Align feature distributions from multiple source domains into a shared space.
  - **Algorithm:** Minimize classification loss while adversarially confusing a domain discriminator to enforce domain-invariant representations.
- **Single-side Domain Alignment**
  - **Idea:** Align only real samples across domains to avoid fake-sample diversity interference.
  - **Algorithm:** Apply domain alignment constraints to live samples while keeping spoof samples separated.
- **Adaptive Reweighting Domain Generalization**
  - **Idea:** Assign adaptive weights to samples based on domain information during alignment.
  - **Algorithm:** Estimate domain-dependent weights and incorporate them into alignment loss.
- **Meta-learning based FAS**
  - **Idea:** Simulate domain shift during training to improve generalization.
  - **Algorithm:**
    1. Split source domains into meta-train and meta-test.
    2. Update model on meta-train domains.
    3. Optimize model to perform well on meta-test domains.
    4. Iterate to learn domain-robust parameters.

3.2.2 Batch-Instance Normalization

IBN-Net (Non-parametric Combination)

- **Idea:** Combine BN and IN structurally by assigning IN to shallow layers (reduce style variance) and BN to deeper layers (retain semantics).
- **Algorithm:**

  - Split feature channels into two groups.
  - Apply IN to one group and BN to the other.
  - Concatenate outputs along channel dimension.
- **Limitation:** Fixed structural design; no adaptive balancing across samples.

Batch-Instance Normalization (BIN)

- **Idea:** Learn a channel-wise balance between BN and IN outputs to adaptively trade off discrimination and invariance.
- **Algorithm:**
  - Compute normalized outputs:

$$X_{BN} = \mathrm{BN}(X), \quad X_{IN} = \mathrm{IN}(X)$$

  - Learn channel-wise parameter $\alpha \in [0, 1]$.
  - Fuse:

$$Y = \alpha X_{BN} + (1 - \alpha) X_{IN}$$

- **What BIN inherits:**
  - Discriminative capability from BN.
  - Style-invariance from IN.
- **What BIN solves:**
  - Reduces reliance on purely batch statistics.
  - Provides learnable trade-off instead of fixed structural combination.
- **Limitation:** The balance factor α is shared across samples, which may limit adaptability under large domain diversity.

Meta Batch-Instance Normalization

- **Idea:** Improve generalization by optimizing normalization parameters under simulated domain shift using meta-learning.
- **Algorithm:**
  - Split domains into meta-train and meta-test.
  - Update normalization parameters on meta-train.
  - Optimize performance on meta-test.
- **Limitation:** Still relies on shared balancing parameters across samples.

4 Methodology: Adaptive normalized representation

4.1 Proposed method: Adaptive feature normalization

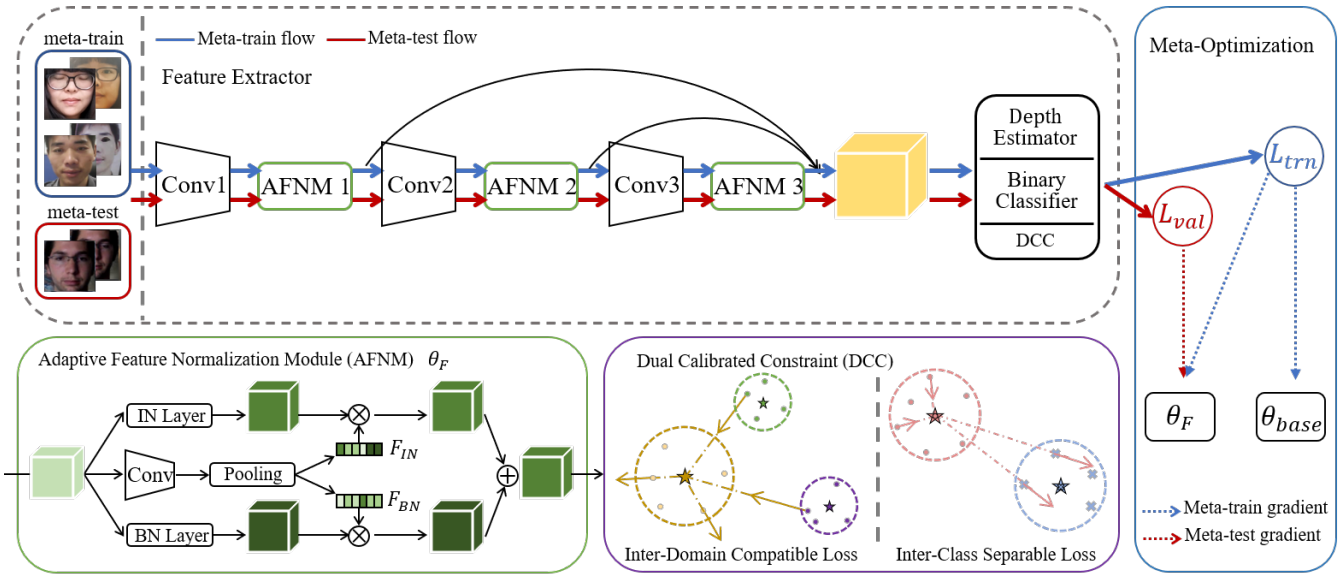

Figure 2: The overall framework of the Adaptive Normalized Representation Learning (ANRL) model, illustrating the episodic meta-learning process and the Adaptive Feature Normalization Module (AFNM).

The core method proposed in the research paper is the **Adaptive normalized representation learning (ANRL)** framework [10]. The authors identify a fundamental trade-off in normalization techniques used for Domain generalization (DG): Batch normalization (BN) preserves discriminative information but is fragile to domain shifts, whereas Instance normalization (IN) eliminates domain style but may discard discriminative cues.

To resolve this, the paper introduces the **Adaptive feature normalization module (AFNM)**. Unlike prior methods [13] that use fixed parameters to combine normalizations, AFNM estimates sample-wise balance factors to adaptively fuse features extracted by BN and IN.

4.1.1 Normalization

In this section, we formalize batch normalization and instance normalization with 4D tensor $X \in \mathbb{R}^{B\times C\times H\times W}$, where B is batch size, C channels, and $H \times W$ spatial dimensions.

Batch normalization (BN).

$$\mu_c^{\mathrm{BN}} = \frac{1}{BHW}\sum_{b=1}^{B}\sum_{i=1}^{H}\sum_{j=1}^{W} X_{b,c,i,j}, \quad (\sigma_c^{\mathrm{BN}})^2 = \frac{1}{BHW}\sum_{b,i,j}\left(X_{b,c,i,j} - \mu_c^{\mathrm{BN}}\right)^2. \tag{15}$$

The normalized output is then

$$\hat{X}^{\mathrm{BN}}_{b,c,i,j} = \frac{X_{b,c,i,j} - \mu^{\mathrm{BN}}_c}{\sqrt{(\sigma^{\mathrm{BN}}_c)^2 + \epsilon}}, \tag{16}$$

followed by a learned affine transformation

$$Y^{\mathrm{BN}}_{b,c,i,j} = \gamma_c \hat{X}^{\mathrm{BN}}_{b,c,i,j} + \beta_c. \tag{17}$$

Because μ^{BN}_c and σ^{BN}_c depend on the mini-batch, BN encourages shared statistics across samples and improves stability, but it can be sensitive to domain shifts when batch composition changes.

Instance normalization (IN).

$$\mu^{\mathrm{IN}}_{b,c} = \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W} X_{b,c,i,j}, \quad (\sigma^{\mathrm{IN}}_{b,c})^2 = \frac{1}{HW}\sum_{i,j}\left(X_{b,c,i,j} - \mu^{\mathrm{IN}}_{b,c}\right)^2, \tag{18}$$

and

$$\hat{X}^{\mathrm{IN}}_{b,c,i,j} = \frac{X_{b,c,i,j} - \mu^{\mathrm{IN}}_{b,c}}{\sqrt{(\sigma^{\mathrm{IN}}_{b,c})^2 + \epsilon}}, \quad Y^{\mathrm{IN}}_{b,c,i,j} = \gamma_c \hat{X}^{\mathrm{IN}}_{b,c,i,j} + \beta_c. \tag{19}$$

IN removes instance-specific style and illumination variations because normalization is independent across samples. This improves robustness to domain shift but may discard discriminative cues that correlate with identity or attack artifacts.

4.1.2 Methodology

Let $X \in \mathbb{R}^{C\times H\times W}$ be the input feature map. The AFNM operates via the following mathematical logic:

1. **Dual normalization**: The input is processed in parallel to generate X_c^{BN} and X_c^{IN}.
2. **Statistic extraction**: A global statistic vector $S \in \mathbb{R}^C$ is computed via global average pooling (F_{gp}) to capture channel-wise information:

$$S_c = F_{gp}(X) = \frac{1}{H \times W}\sum_{i=1}^{H}\sum_{j=1}^{W} X_c(i,j) \tag{20}$$

3. **Factor estimation**: A compact representation $Z \in \mathbb{R}^d$ is learned via a fully connected layer (F_{fc}) with weights W and ReLU activation δ:

$$Z = F_{fc}(S) = \delta(WS) \tag{21}$$

4. **Adaptive fusion**: Soft attention weights for BN (B) and IN (I) are computed using Sigmoid

functions σ. The normalized balance factor α_c for channel c is derived to determine the importance of the BN feature relative to the IN feature:

$$\alpha_c = \frac{B_c}{B_c + I_c} = \frac{\sigma(W_B Z)_c}{\sigma(W_B Z)_c + \sigma(W_I Z)_c} \tag{22}$$

The final representation Y is a weighted combination, allowing the network to rely more on IN for samples with high domain bias and BN for samples where discriminative information is paramount:

$$Y_c = \alpha_c X_c^{BN} + (1 - \alpha_c) X_c^{IN} \tag{23}$$

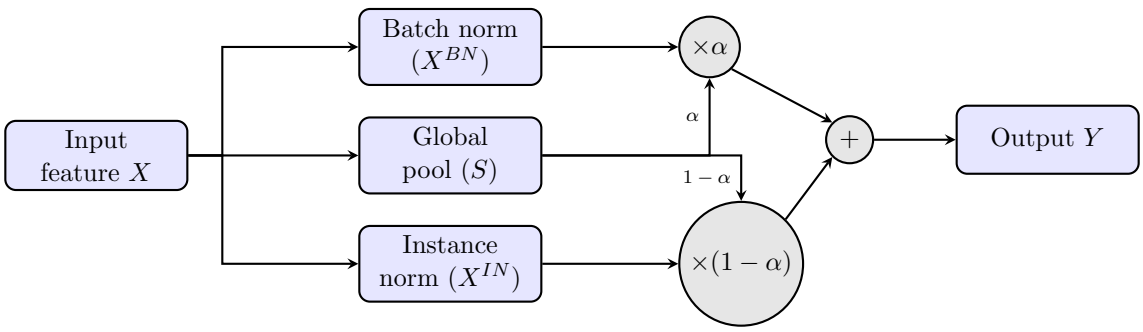

Figure 3: Logic flow of the AFNM, illustrating the derivation of sample-wise weights α to fuse X^{BN} and X^{IN}.

4.2 Technical objectives and adopted approach

4.2.1 Technical objectives

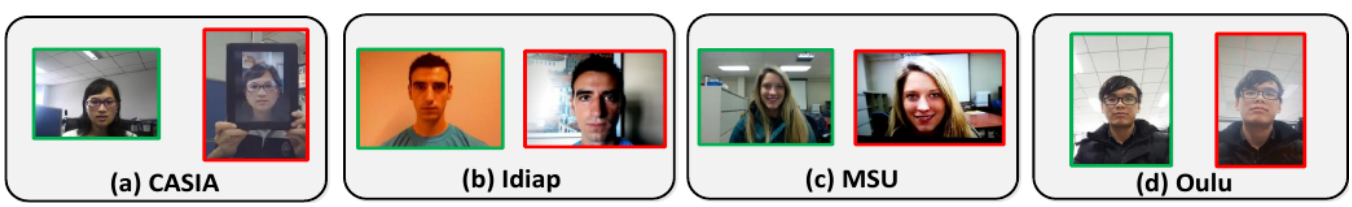

Figure 4: Samples from CASIA-MFSD, Idiap, MSU, and Oulu datasets. Variations in illumination, background, and materials are the severe domain shift.

The primary objective is to learn a **Generalizable representation** for Face anti-spoofing that performs well on unseen scenarios (target domains). Specifically, the method aims to construct a feature space that is:

- **Domain-agnostic:** Invariant to domain shifts such as changes in illumination, camera sensors, or background.
- **Discriminative:** Capable of distinguishing between live faces and varied presentation attacks.

4.2.2 Adopted approach: Dual calibrated constraints (DCC)

To guide AFNM in learning optimal balance factors α_c, the authors propose the **Dual calibrated constraints (DCC)**, consisting of two specific loss functions employed during a meta-learning optimization process.

1. Inter-domain compatible (IDC) loss:

To mitigate the domain bias inherent in BN features, this loss aims to align distributions across different source domains. It minimizes the distance between samples from different domains (inter-domain) while maximizing the variance within the same domain (intra-domain) to promote domain mixing.

$$\mathcal{L}_{IDC} = \sum_{k=1}^{K} (D_{dd}^k - D_{sd}^k) \tag{24}$$

where D_{dd}^k represents the distance between samples in domain k and centroids of other domains, and D_{sd}^k represents the distance between samples in domain k and their own domain centroid.

Note: Domain distances are minimized; we want to reduce BN corruption because of domain bias, which clusters each type of attack (e.g., video replay, print attack, etc.).

2. Inter-class separable (ICS) loss:

To prevent the loss of discriminative information (a risk when using IN), this loss enforces separation between real and fake classes. It compacts real samples around a center $\widehat{C_r}$ while ensuring fake samples are distant from it.

$$\mathcal{L}_{ICS} = D_{rr} - D_{rf} - D_{fr} \tag{25}$$

where D_{rr} is intra-class real distance, D_{rf} is distance between real samples and fake centroids, and D_{fr} is distance between fake samples and the real centroid.

Note: D_{ff} is not included because presentation attacks are highly diverse; forcing them into a single cluster would harm generalization.

4.3 Optimization strategy and base model supervision

The ANRL framework employs a dual-level optimization strategy to learn a domain-agnostic and discriminative representation. This involves updating the base model parameters (θ_{base}) using standard supervision, while the Adaptive Feature Normalization Module (AFNM) parameters (θ_F) are updated via an episodic meta-learning paradigm.

4.3.1 Base model supervision: classification and depth estimation

The base model consists of three primary components: a Feature Extractor (Ext), a Binary Classifier (BC), and an auxiliary Depth Estimator (Dep). During training, the base model is governed by two fundamental loss functions: $\mathcal{L}_{Cls}$ and $\mathcal{L}_{Dep}$.

1. Binary classification ($\mathcal{L}_{Cls}$). Face anti-spoofing is inherently a binary classification task. The binary classifier is optimized using a standard cross-entropy loss formulation:

$$\mathcal{L}_{Cls} = -\sum_{(x_i, y_i)} y_i \log(BC(Ext(x_i))) \tag{26}$$

where x is the input image tensor (integrating RGB and HSV color channels) and y is the ground-truth liveness label. While effective for separating classes, optimizing solely on $\mathcal{L}_{Cls}$ often causes deep neural networks to overfit to domain-specific 2D textures, resulting in poor cross-domain generalization.

2. Depth estimation ($\mathcal{L}_{Dep}$). To prevent the feature extractor from converging on superficial 2D artifacts, geometric supervision is introduced. Fixed faces possess intrinsic 3D spatial depth, whereas spoofing attacks (e.g., printed photos or video replays) are geometrically flat planes.
The depth estimator is tasked with predicting a topographical depth map for live faces and a flat zero-map for spoof faces. This is enforced using a pixel-wise Mean Squared Error (MSE) loss:

$$\mathcal{L}_{Dep} = \sum_{(x_i, dep_i)} \|Dep(Ext(x_i)) - dep_i\|_2^2 \tag{27}$$

where *dep* is the pseudo-depth map. Specifically, for genuine faces, *dep* is a dense 3D map calculated using PRNet [4], while for spoofing attacks, *dep* is a flat zero-map. By minimizing $\mathcal{L}_{Dep}$, the feature extractor is forced to mine 3D structural cues, enriching the discriminative capacity of the representation space.

4.3.2 Optimization via meta-learning

To optimize the adaptive fusion of normalizations, the framework uses **meta-learning** to simulate domain shifts during training. The available source domains are split into **distinct** meta-train (D_{trn}) and meta-test (D_{val}) sets.
The base parameters (θ_{base}) are updated using the standard combined loss ($\mathcal{L}_{base} = \mathcal{L}_{Cls} + \mathcal{L}_{Dep}$). Conversely, AFNM parameters (θ_F) undergo a simulated domain shift. They are first temporarily updated on D_{trn} using the Dual Calibrated Constraints ($\mathcal{L}_{IDC} + \mathcal{L}_{ICS}$) alongside the base losses. Crucially, the actual parameter update for θ_F incorporates the gradients derived from the loss evaluated on the unseen D_{val} domains. This forces the AFNM to learn a normalization strategy that explicitly generalizes to out-of-distribution data.

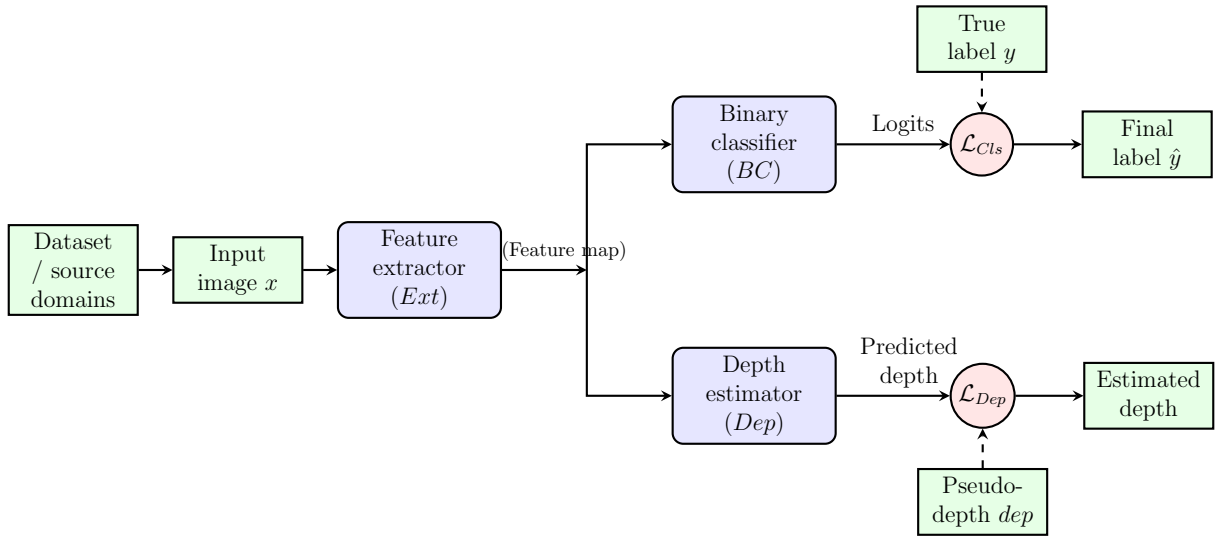

Figure 5: Architecture of the base model illustrating the dual supervision strategy. The feature extractor is simultaneously guided by the classification loss ($\mathcal{L}_{Cls}$) and the physically-grounded auxiliary depth estimation loss ($\mathcal{L}_{Dep}$).

Algorithm 1 The optimization strategy of ANRL

Require: N source domains $\mathcal{D} = [D_1, D_2, \ldots, D_N]$

Initialize parameters θ_F of AFNM and parameters θ_{base} of other modules.
Determine learning rates β_1, β_2 and hyper-parameters λ_1, λ_2.
Shuffle all samples from different domains.
for $t = 1$ to N_{epoch} **do**
    Normal train: Sample batch in $\mathcal{D}$.
    $\mathcal{L}_{base}(\theta_{base}, \theta_F) = \sum_{\mathcal{D}} \mathcal{L}_{Cls} + \mathcal{L}_{Dep}$.
    $\theta_{base} \leftarrow \theta_{base} - \beta_1 \nabla_{\theta_{base}} \mathcal{L}_{base}(\theta_{base}, \theta_F)$.
    Meta-train: Sample batch in meta-train domains D_{trn}.
    $\mathcal{L}_{trn}(\theta_{base}, \theta_F) = \mathcal{L}_{Cls(D_{trn})} + \mathcal{L}_{Dep(D_{trn})} + \lambda_1 \mathcal{L}_{IDC(D_{trn})} + \lambda_2 \mathcal{L}_{ICS(D_{trn})}$.
    $\theta'_F = \theta_F - \beta_1 \nabla_{\theta_F} \mathcal{L}_{trn}(\theta_{base}, \theta_F)$.
    Meta-test: Sample batch in meta-test domains D_{val}.
    $\mathcal{L}_{val}(\theta_{base}, \theta'_F) = \mathcal{L}_{Cls(D_{val})} + \mathcal{L}_{Dep(D_{val})} + \lambda_1 \mathcal{L}_{IDC(D_{val})} + \lambda_2 \mathcal{L}_{ICS(D_{val})}$.
    Meta-optimization: Update AFNM using combined gradients:
    $\theta_F \leftarrow \theta_F - \beta_2 \nabla_{\theta_F} (\mathcal{L}_{trn}(\theta_{base}, \theta_F) + \mathcal{L}_{val}(\theta_{base}, \theta'_F))$.
end for
return Model parameters θ_F and θ_{base}.

4.3.3 Effectiveness analysis

The method's effectiveness lies in its granular handling of domain information. While previous methods used shared parameters for normalization fusion, ANRL's sample-wise factors allow it to "adaptively select" the best normalization per image.

Experiments demonstrate that ANRL outperforms state-of-the-art methods (such as SSDG [7] and RFM [17]) on challenging cross-dataset protocols (e.g., OULU, CASIA, Idiap, MSU).

5 Conclusion

5.1 Evaluation

5.1.1 Strengths

1. Study effects of normalization

Previous domain generalization techniques in deep learning often rely on static hybrid normalizations, utilizing shared, non-parametric rules (e.g., adding Instance Normalization at specific layers) or fixed learnable parameters to fuse Batch Normalization (BN) and Instance Normalization (IN). The primary strength of the Adaptive Feature Normalization Module (AFNM) is its sample-wise adaptability.

$$Y = \alpha X^{BN} + (1 - \alpha) X^{IN} \tag{28}$$

It allows the model to dynamically prioritize IN when a sample has heavy domain bias, or BN for samples where domain shift is negligible.

2. Asymmetric class constraints for spoof detection

Dual Calibrated Constraints (DCC) lies in Inter-Class Separable (ICS) loss. Compared to normal machine learning algorithms that typically attempt to cluster all instances of a specific class tightly around a centroid, the ANRL framework correctly assumes that while genuine faces share a common physiological distribution, presentation attacks (spoofs) are highly heterogeneous (e.g., printed paper, video replay, 3D masks).

3. Meta-learning optimization strategy

The framework employs an episodic training paradigm that simulates real-world domain shifts during the training phase. By dividing the available source domains into meta-train (D_{trn}) and meta-test (D_{val}) sets, the optimization of the AFNM parameters (θ_F) is forced to generalize. The parameters are updated using the combined gradients from both sets:

$$\theta_F \leftarrow \theta_F - \beta_2 \nabla_{\theta_F} (\mathcal{L}_{trn} + \mathcal{L}_{val}) \tag{29}$$

This *learns how to normalize* robustly across domain boundaries.

- Performance: higher HTER, AUC score than GAN methodology and hand-extracted features.

5.1.2 Limitations

- Train datasets dependency: The framework relies on an episodic meta-learning strategy that intrinsically requires data from multiple, distinct source domains to simulate realistic domain shifts. Consequently, it does not work well or generalize effectively if the available source datasets are too limited in diversity.

- Performance ceiling: Because the method relies purely on spatial features derived from 2D images and a pseudo-depth geometric map, its classification performance may be less than state-of-the-art multi-modal methods (e.g., Vision-Language Models) that leverage richer contextual data streams.
- Spatial-only classification: The model only uses spatial (image) data for classification. It lacks a temporal domain mechanism (e.g., audio, video sequence analysis), which means it ignores temporal consistency cues that are highly critical for detecting sophisticated video replay attacks.
- Auxiliary supervision vulnerability: The method heavily relies on an auxiliary Depth Estimator supervised by pseudo-depth maps. Any inherent errors or biases generated by the external depth estimator (like PRNet) will propagate into the main feature extractor.
- Asymmetric compactness assumption failure: The Inter-Class Separable loss forces genuine faces into a highly compact cluster (D_{rr}). If a target domain introduces live faces with extreme structural variance (e.g., heavy occlusion or extreme pose), they may fall outside this compact distribution and be falsely rejected as spoofing attacks.

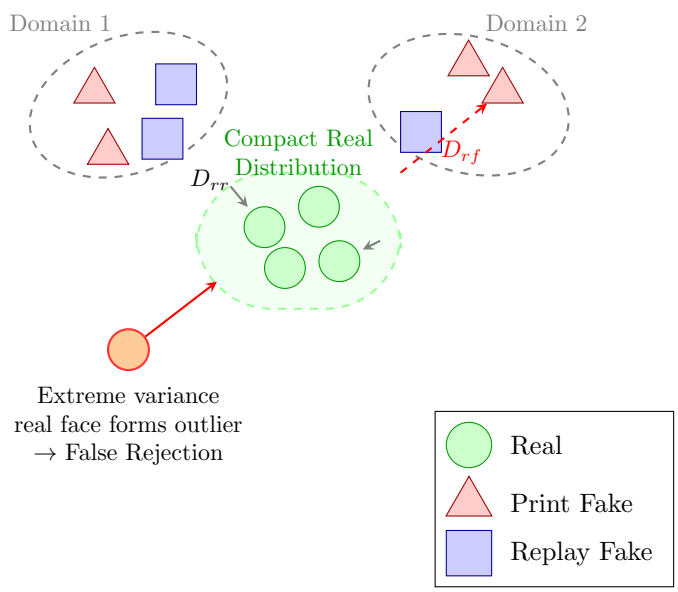

Figure 6: Visualization of the asymmetric class assumption limitation. The framework strongly clusters real faces (D_{rr}) while scattering fake ones. Extreme structural variants of real faces risk falling outside the cluster, triggering false rejections.

5.1.3 Applicability

- Easy to implement: The framework can be seamlessly integrated using deep learning libraries like PyTorch, operating efficiently on standard architectures such as the RFM network base.
- Robust out-of-the-box generalization: Thanks to its domain-agnostic feature extraction and simulated domain-shift training, the method excels when deployed in completely unseen scenarios (e.g., unseen cameras, lighting, and backgrounds), requiring no target-domain fine-tuning.

- Security-critical deployments: This methodology is highly applicable to unconstrained environments where lighting and sensors fluctuate wildly, such as mobile banking authentication, physical access control verification, and general smartphone unlocking mechanisms.

5.2 Improvements

5.2.1 Personal Insights on Effectiveness and Practical Relevance

Effectiveness: ANRL dynamically combats extreme variance between physical distortions (moiré) and digital tampering (deepfakes) by adaptively blending batch and instance normalizations based on input samples.

Practical Relevance: Visualization Method [6]: By explicitly visualizing data domain changes in the CNN, practitioners can dynamically track how the Dual Calibrated Constraints ($\mathcal{L}_{IDC}$ and $\mathcal{L}_{ICS}$) across epochs, allowing to fine-tune the AFNM's balance factors (α), moving beyond static post-training t-SNE diagrams.

5.2.2 Extensions

Extension 1: Simulating Spoofing Clues (Augmentation) [5]. To overcome reliance on diverse source domains, ANRL can be extended with Joint Physical-Digital augmentation. During Meta-Train, genuine faces are augmented into simulated attacks (X_{sim}) using Simulated Physical (e.g., color distortion, θ_{SPSC}) and Digital (e.g., Gaussian blur, θ_{SDSC}) Spoofing Clues.

$$X_{sim} = Aug(X_{real}; \theta_{SPSC}, \theta_{SDSC}) \tag{30}$$

Feeding these synthesized attacks into the AFNM forces the meta-learning optimizer to generalize across a vastly expanded domain space, drastically improving robustness against unseen out-of-distribution threats.

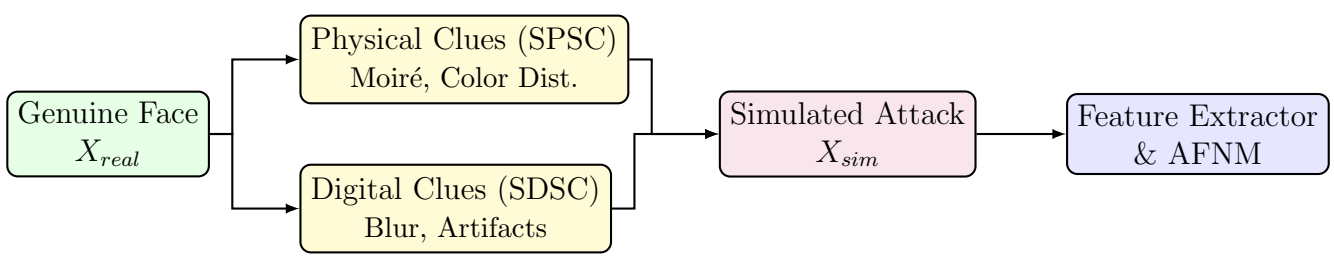

Figure 7: Conceptual workflow of the Joint Physical-Digital Augmentation extension. Genuine faces are augmented with simulated physical and digital spoofing clues before being processed by the AFNM.

Extension 2: Temporal-Spectral ANRL (TS-ANRL). While ANRL excels in 2D spatial generalization, practical FAS heavily relies on temporal cues (e.g., eye blinking). Extending AFNM

with 3D convolutions allows temporal-IN to filter dynamic lighting fluctuations, while temporal-BN preserves chronological inconsistencies inherent to video replay attacks.

References

[1] A. Anjos and S. Marcel. Can face anti-spoofing countermeasures work in a real world scenario? Technical Report, Idiap Research Institute, 2013. Includes motion difference ROI based countermeasure description. URL: https://www.idiap.ch/~aanjos/papers/icb-2013-1.pdf.

[2] T. de Freitas Pereira, A. Anjos, J. M. D. Martino, and S. Marcel. Lbp-top based countermeasure against face spoofing attacks. In *Proceedings of the International Workshop on Computer Vision with Local Binary Pattern Variants*, ACCV Workshops, pages 1–12. Springer, November 2012. doi:10.1007/978-3-642-37410-4_11.

[3] S. Dhamija, D. Gupta, and M. Sachdeva. Face liveness detection using challenge response mechanism based on ear and mar. *International Journal of Computer Networks & Communications (IJCNC)*, 11(2):45–60, 2019. Uses Mouth Aspect Ratio (MAR) and Eye Aspect Ratio (EAR) for challenge-response face anti-spoofing.

[4] Y. Feng, F. Wu, X. Shao, Y. Wang, and X. Zhou. Joint 3d face reconstruction and dense alignment with position map regression network. In *Proceedings of the European conference on computer vision (ECCV)*, pages 534–551, 2018.

[5] X. He, D. Liang, S. Yang, Z. Hao, H. Ma, B. Mao, X. Li, Y. Wang, P. Yan, and A. Liu. Joint physical-digital facial attack detection via simulating spoofing clues. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 995–1004, 2024.

[6] M. Huang, C. Nie, and W. Zhong. A visualization method for data domain changes in cnn networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 964–972, 2024.

[7] Y. Jia, J. Zhang, S. Shan, and X. Chen. Single-side domain generalization for face anti-spoofing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8484–8493, 2020.

[8] K. Kollreider, H. Fronthaler, and J. Bigün. Non-intrusive liveness detection by face images. *Image and Vision Computing*, 27(3):231–237, 2009.

[9] S. Liu, Z. Feng, J. Zhao, and Y. Zhang. Face anti-spoofing based on deep learning: A comprehensive survey. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 3(1):14–33, 2021. doi:10.1109/TBIOM.2020.3010551.

[10] S. Liu, K.-Y. Zhang, T. Yao, M. Bi, S. Ding, J. Li, F. Huang, and L. Ma. Adaptive normalized representation learning for generalizable face anti-spoofing. *arXiv preprint arXiv:2108.02667*, 2021. doi:10.48550/arXiv.2108.02667.

[11] Y. Liu and X. Tan. An anti-photo spoof method in face recognition based on the analysis of fourier spectra with sparse logistic regression. In *Proceedings of the 2009 International Conference on Pattern Recognition (CCPR)*, 2009. doi:10.1109/CCPR.2009.5344092.

[12] Z. Ming, M. Visani, M. M. Luqman, and J.-C. Burie. A survey on anti-spoofing methods for facial recognition with rgb cameras of generic consumer devices. *Journal of Imaging*, 6(12):139, 2020. doi:10.3390/jimaging6120139.

[13] H. Nam and H.-E. Kim. Batch-instance normalization for adaptively style-invariant neural networks. In *Advances in neural information processing systems (NeurIPS)*, 2018.

[14] G. Pan, L. Sun, Z. Wu, and S. Lao. Eyeblink-based anti-spoofing in face recognition from a generic webcamera. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Rio de Janeiro, Brazil, October 2007.

[15] G. Pan, L. Sun, Z. Wu, and Y. Wang. Monocular camera-based face liveness detection by combining eyeblink and scene context. *Machine Vision and Applications*, 24(1):215–230, 2011.

[16] A. Seal, S. Ganguly, D. Bhattacharjee, M. Nasipuri, and D. K. Basu. A comparative study of human thermal face recognition based on haar wavelet transform and local binary pattern (lbp). *arXiv preprint arXiv:1309.1009*, 2013.

[17] R. Shao, X. Lan, J. Li, and P. C. Yuen. Regularized fine-grained meta face anti-spoofing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 11974–11981, 2020.

[18] T. Soukupová and J. Čech. Real-time eye blink detection using facial landmarks. In *22nd Computer Vision Winter Workshop (CVWW)*, 2016.